



AI61002: Deep Learning Foundations and Applications

Module 3: Gradient Descent Based Optimization Techniques for Neural Networks

Dr. Somdyuti Paul

Gradient Descent Intuition

We want to iteratively update the cost function $J(\boldsymbol{\theta})$ such that it decreases at every step.

Let the update at any step be along a direction given by $\Delta\boldsymbol{\theta}$.

Using Taylor's series expansion, we have:

$$J(\boldsymbol{\theta} + \alpha\Delta\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \alpha\Delta\boldsymbol{\theta}^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) + \frac{\alpha^2}{2!} \Delta\boldsymbol{\theta}^T \nabla_{\boldsymbol{\theta}}^2 J(\boldsymbol{\theta}) \Delta\boldsymbol{\theta} + \text{h.o.t.}$$

To reduce the cost function, we require:

$$J(\boldsymbol{\theta} + \alpha\Delta\boldsymbol{\theta}) < J(\boldsymbol{\theta})$$

$$\implies J(\boldsymbol{\theta}) + \alpha\Delta\boldsymbol{\theta}^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) < J(\boldsymbol{\theta}) \quad (\text{Using first order approximation for a small value of } \alpha)$$

$$\implies \alpha\Delta\boldsymbol{\theta}^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) < 0$$

$$\implies \Delta\boldsymbol{\theta}^T \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) < 0 \quad (\because \alpha > 0)$$

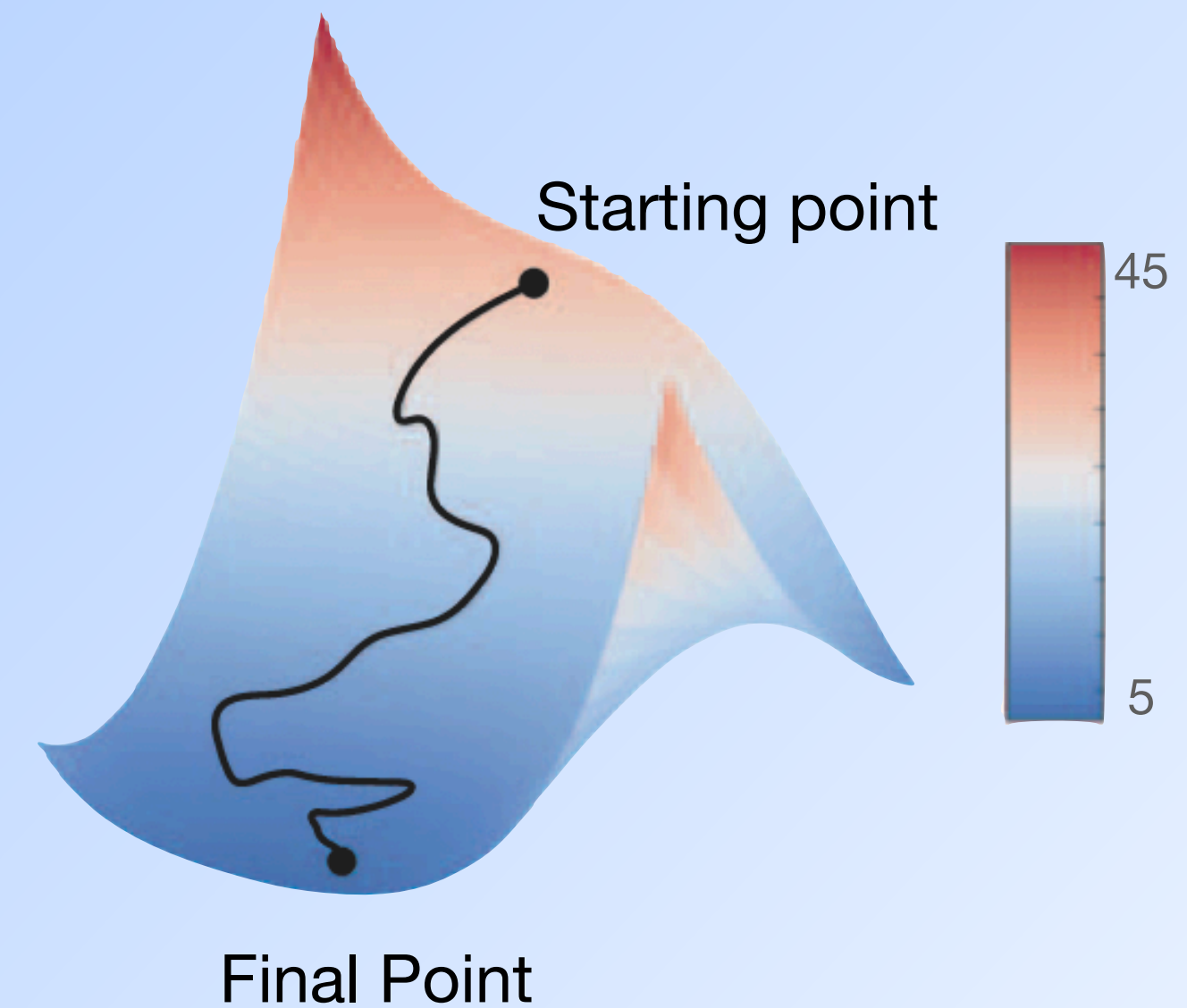
Gradient Descent Intuition

The quantity $\Delta\theta^T \nabla_{\theta} J(\theta)$ is the smallest when
$$\Delta\theta = - \nabla_{\theta} J(\theta)$$

Thus, the negative of the gradient is the direction of steepest descent.

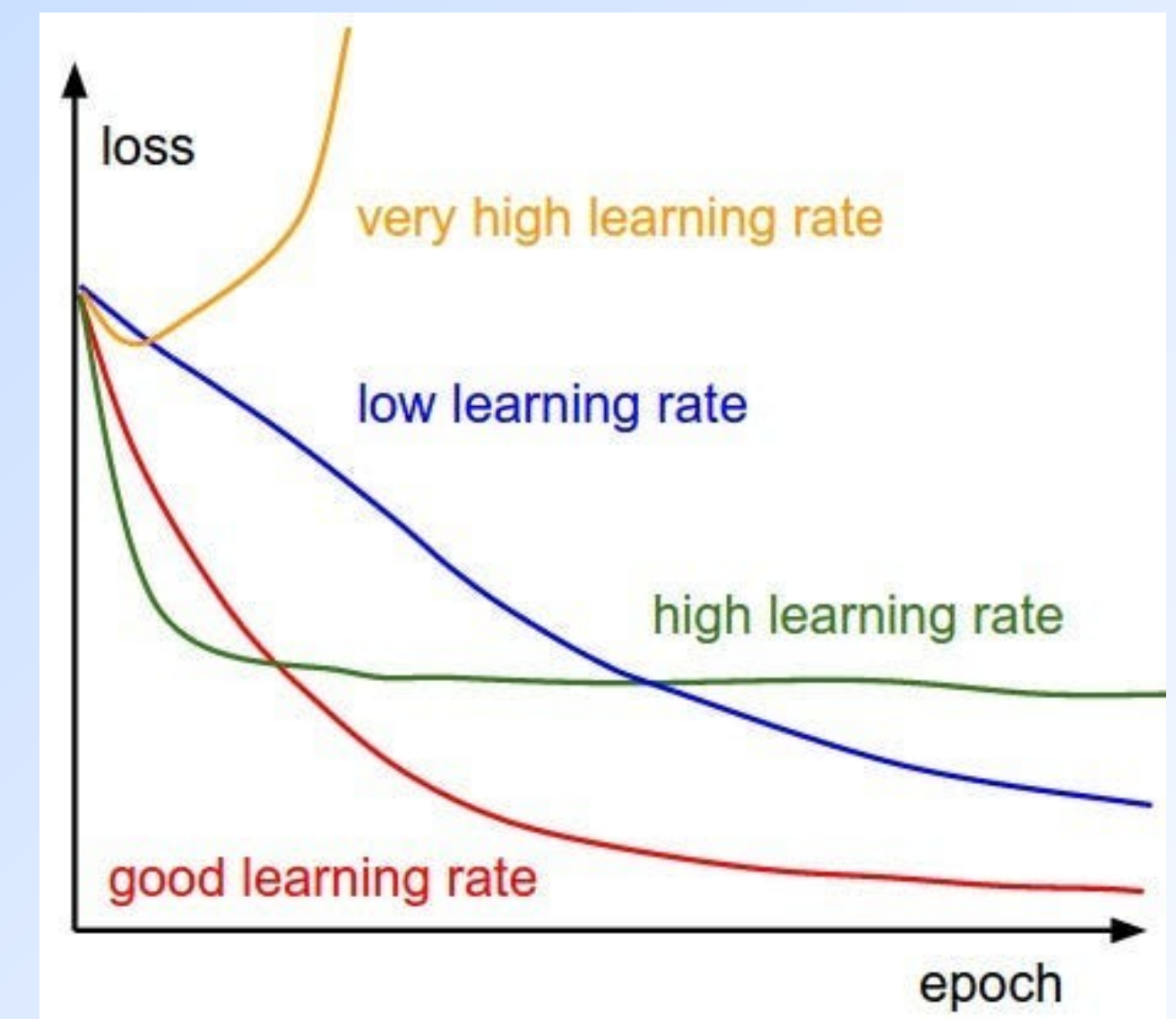
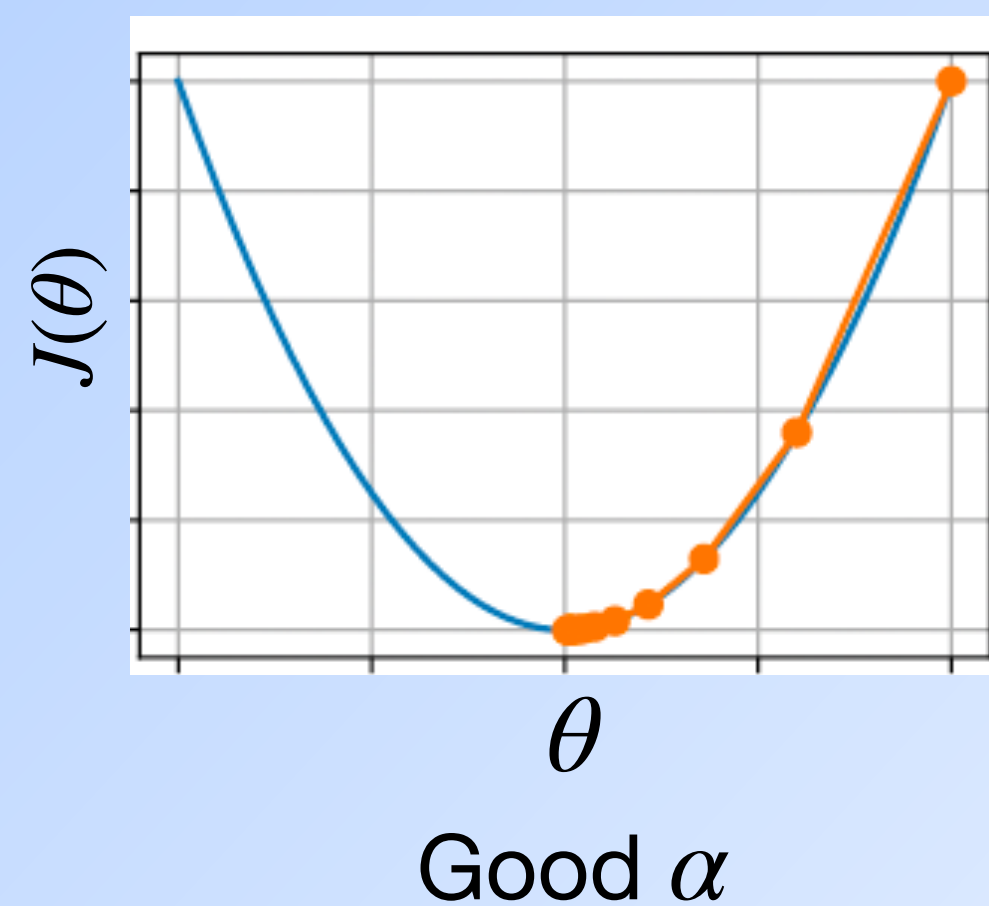
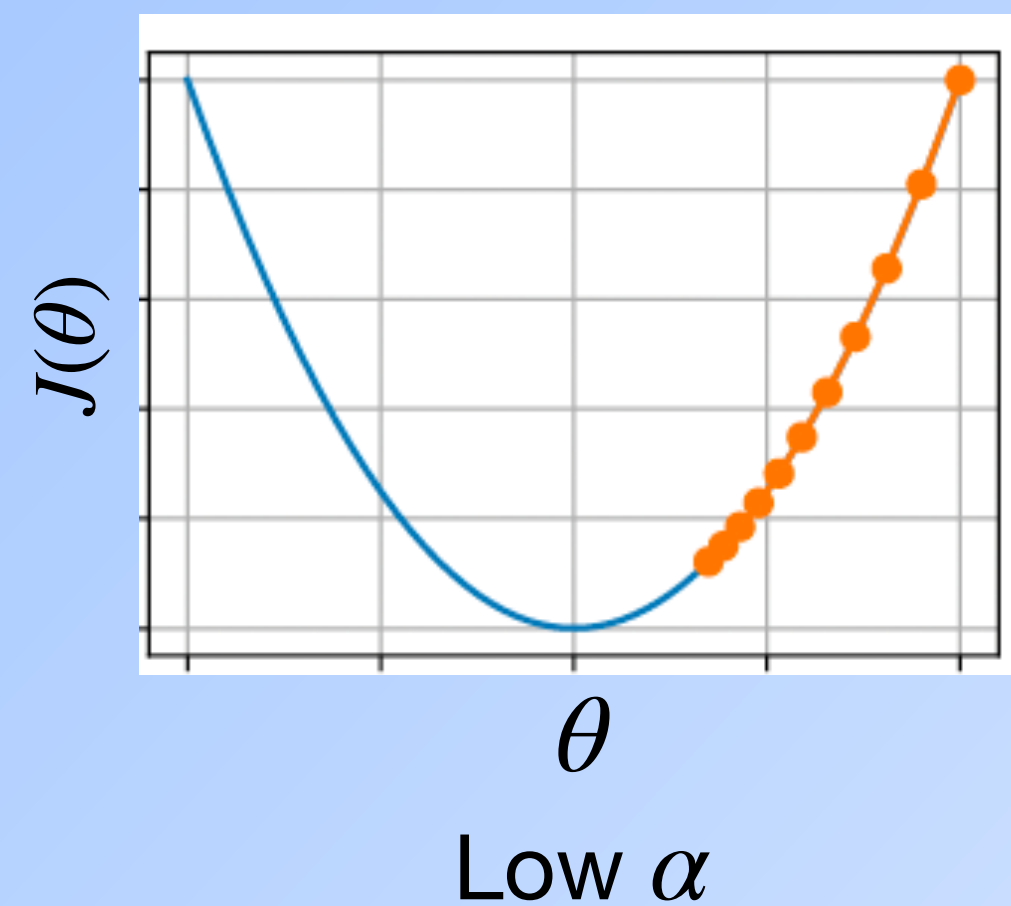
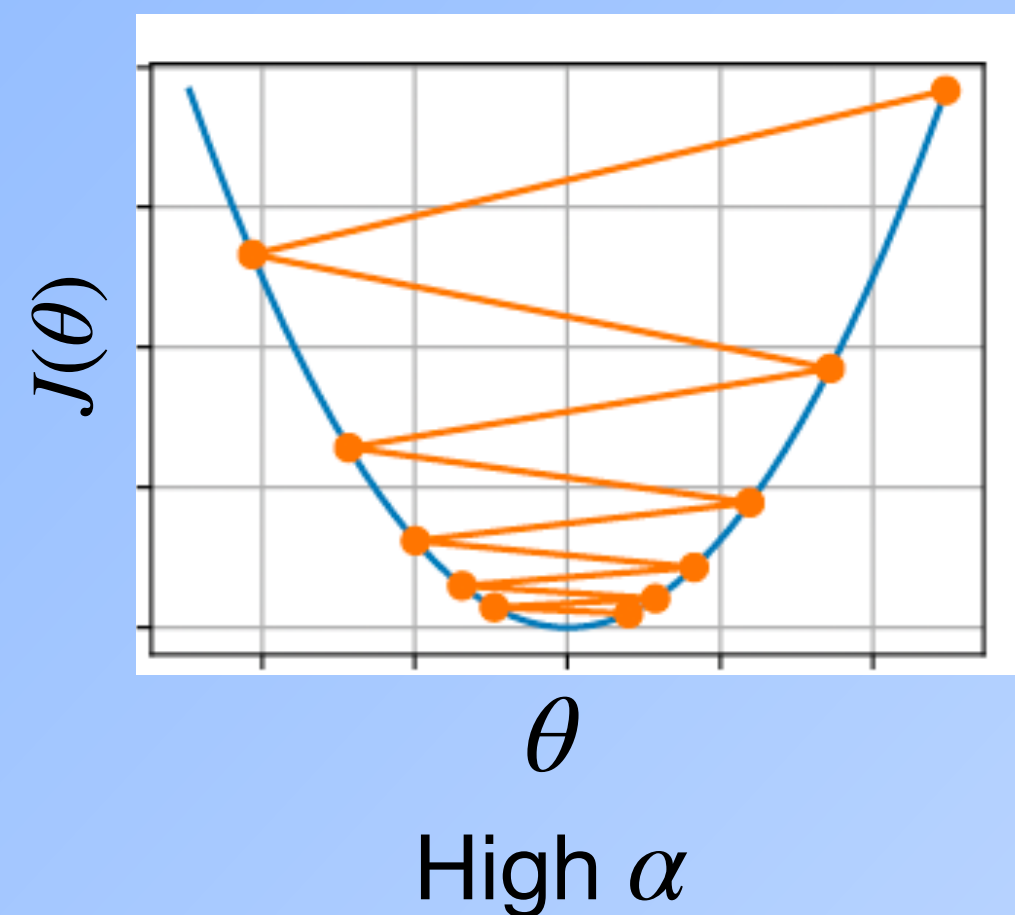
Thereby, the gradient descent (GD) update rule is given by

$$\theta_{\text{new}} = \theta - \alpha \nabla_{\theta} J(\theta)$$



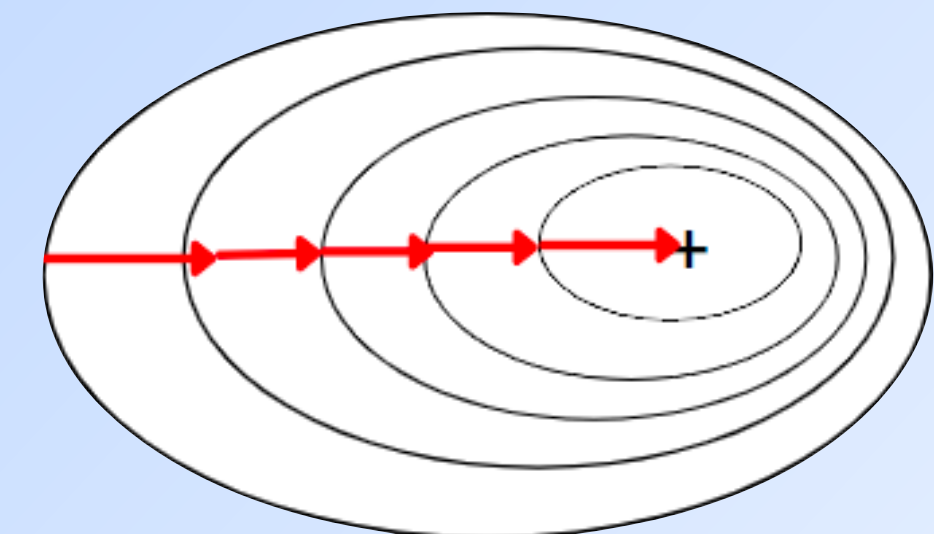
Impact of Learning Rate in Gradient Descent

- Learning rate (α) controls the size of the step taken in the direction of the negative gradient.
- GD with high α may fail to converge due to overshooting of the local minimum
- GD with a very low α converges very slowly.

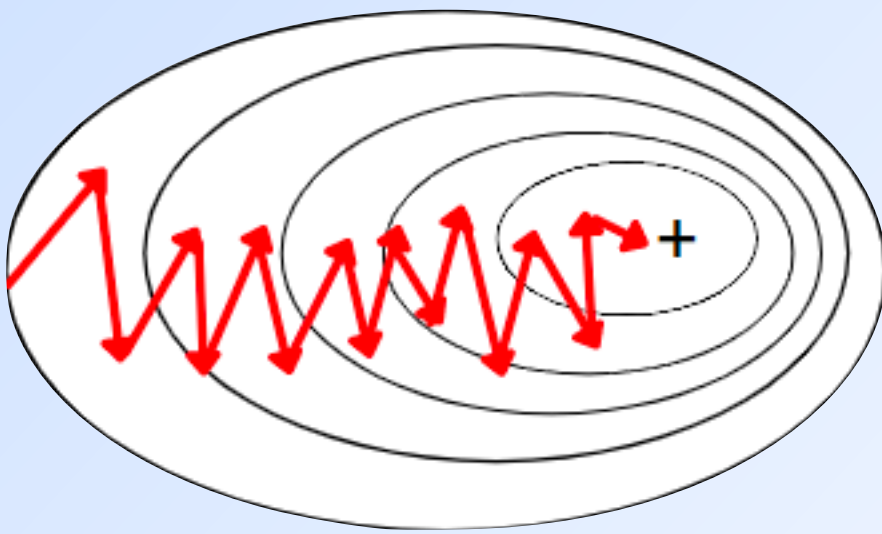


Gradient Descent Variations

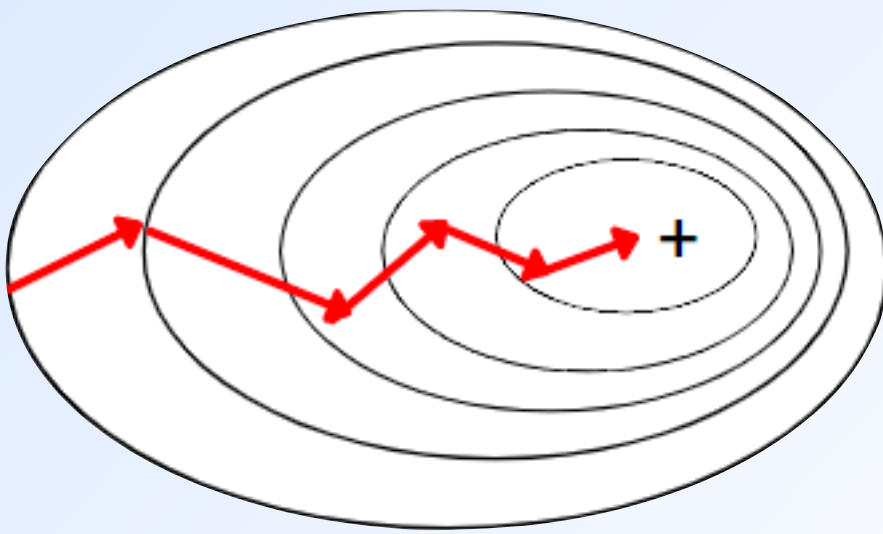
	Batch Gradient Descent	Stochastic Gradient Descent	Minibatch Gradient Descent
Description	Uses the entire training set for gradient computation and parameter updates	Uses individual training samples for gradient computation and parameter updates.	Uses a subset of training samples for gradient computation and parameter updates.
Update rule	$J(\theta) = \frac{1}{M} \sum_{i=1}^M \ell(h_{\theta}(\mathbf{x}^{(i)}), y^{(i)})$ $\theta_{\text{new}} = \theta - \alpha \nabla_{\theta} J(\theta)$	$J(\theta) = \ell(h_{\theta}(\mathbf{x}^{(i)}), y^{(i)})$ $\theta_{\text{new}} = \theta - \alpha \nabla J(\theta)$	$J(\theta) = \frac{1}{B} \sum_{i=1}^B \ell(h_{\theta}(\mathbf{x}^{(i)}), y^{(i)})$ $\theta_{\text{new}} = \theta - \alpha \nabla J(\theta)$ (B is the minibatch size)
Pros	Stable updates	- Less memory requirement - Faster progress	- Updates are more stable than SGD - Lesser memory requirement than batch gradient descent
Cons	- Higher memory requirement - Convergence is slow	Noisy Updates	Requires more memory than stochastic gradient descent.



Batch gradient descent

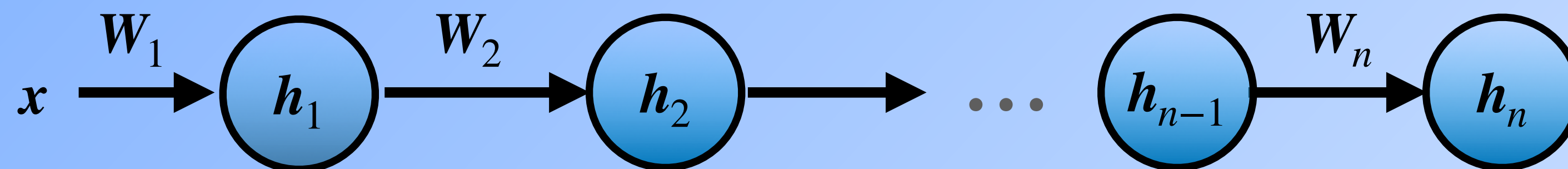


Stochastic gradient descent



Minibatch gradient descent

Vanishing and Exploding Gradients



$$\mathbf{h}_k = f(W_k \mathbf{h}_{k-1})$$

$$\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}} = W_k f'(W_k \mathbf{h}_{k-1})$$

$$\frac{\partial J}{\partial \mathbf{h}_{k-1}} = \frac{\partial J}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$$

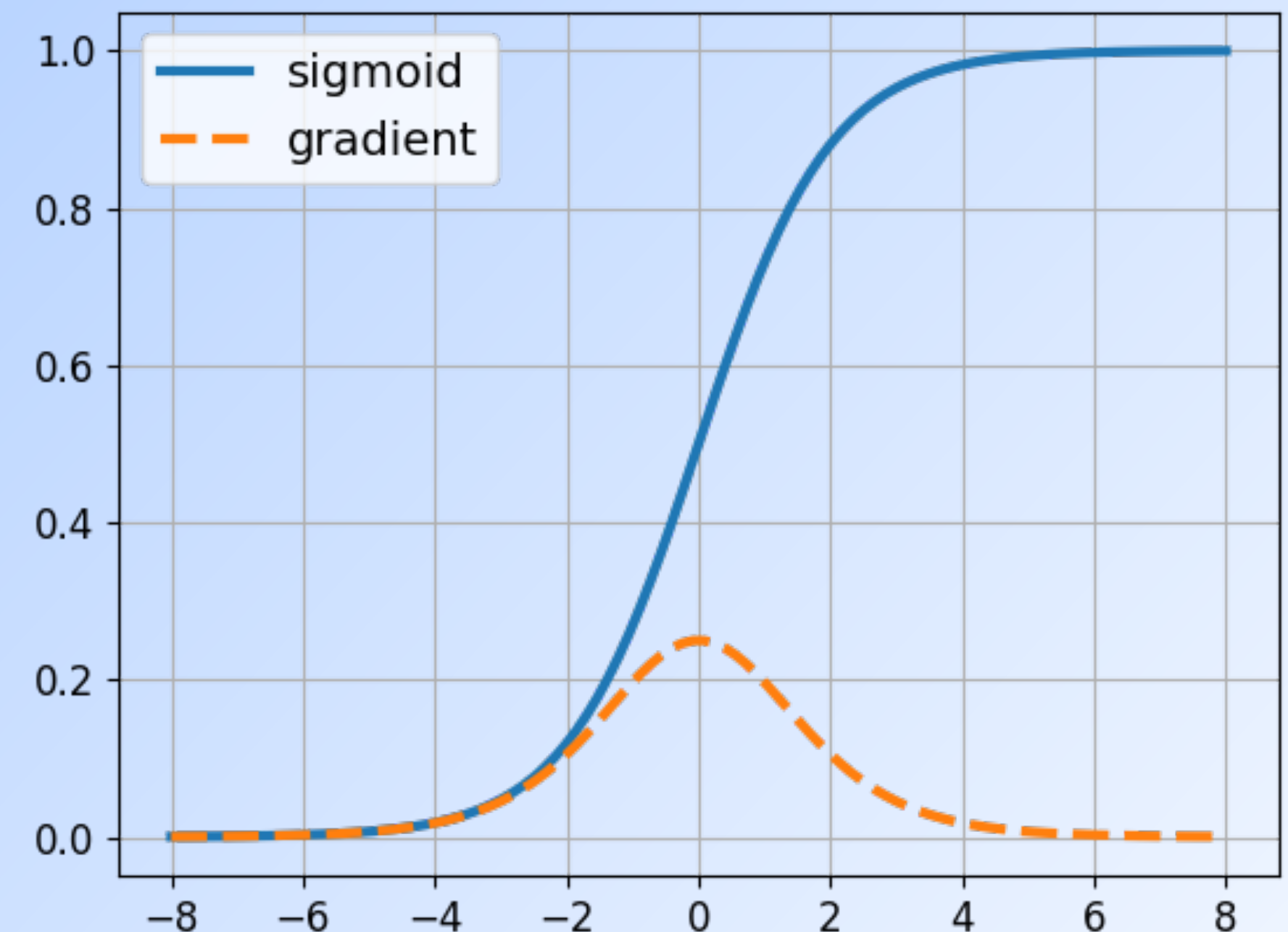
$$= \frac{\partial J}{\partial \mathbf{h}_{k+1}} \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$$

$$= \frac{\partial J}{\partial \mathbf{h}_n} \prod_{i=k}^n \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}}$$

$$= \frac{\partial J}{\partial \mathbf{h}_n} \prod_{i=k}^n W_i f'(W_i \mathbf{h}_{i-1})$$

Vanishing and Exploding Gradients

- The gradient computed at the k^{th} layer is the product of $(n-k+1)$ derivatives of activation functions.
- If the magnitudes of the components of $f'(\mathbf{W}_i \mathbf{h}_{i-1})$ are less than 1, the magnitude of the gradient flowing backwards diminishes progressively.
- Thus, the gradient magnitudes propagated to the earlier layers (for smaller values of k) become negligible, manifesting the vanishing gradient problem.
- Conversely, the gradients flowing backwards explode if the magnitudes of the components of $f'(\mathbf{W}_i \mathbf{h}_{i-1})$ are greater than 1.



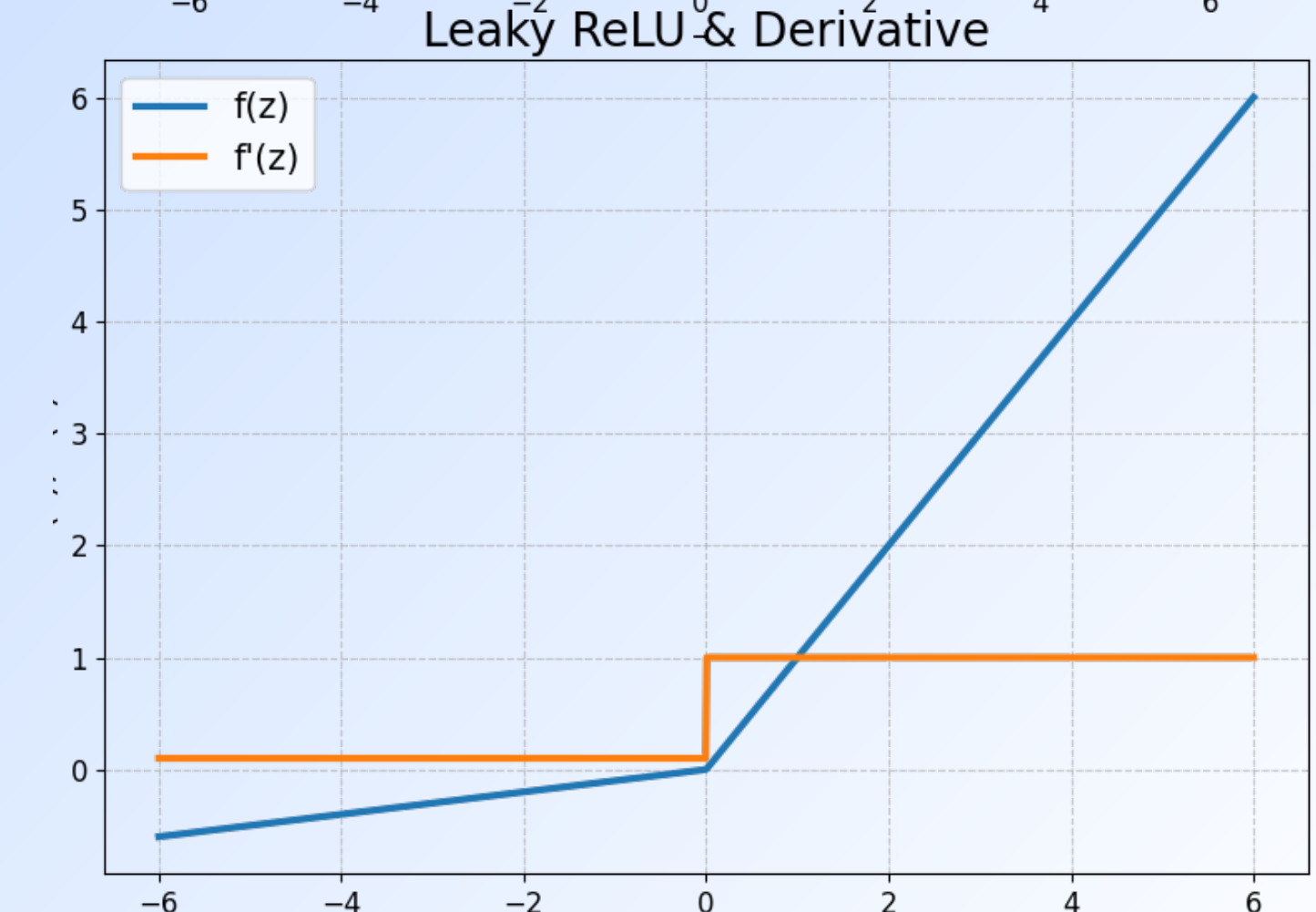
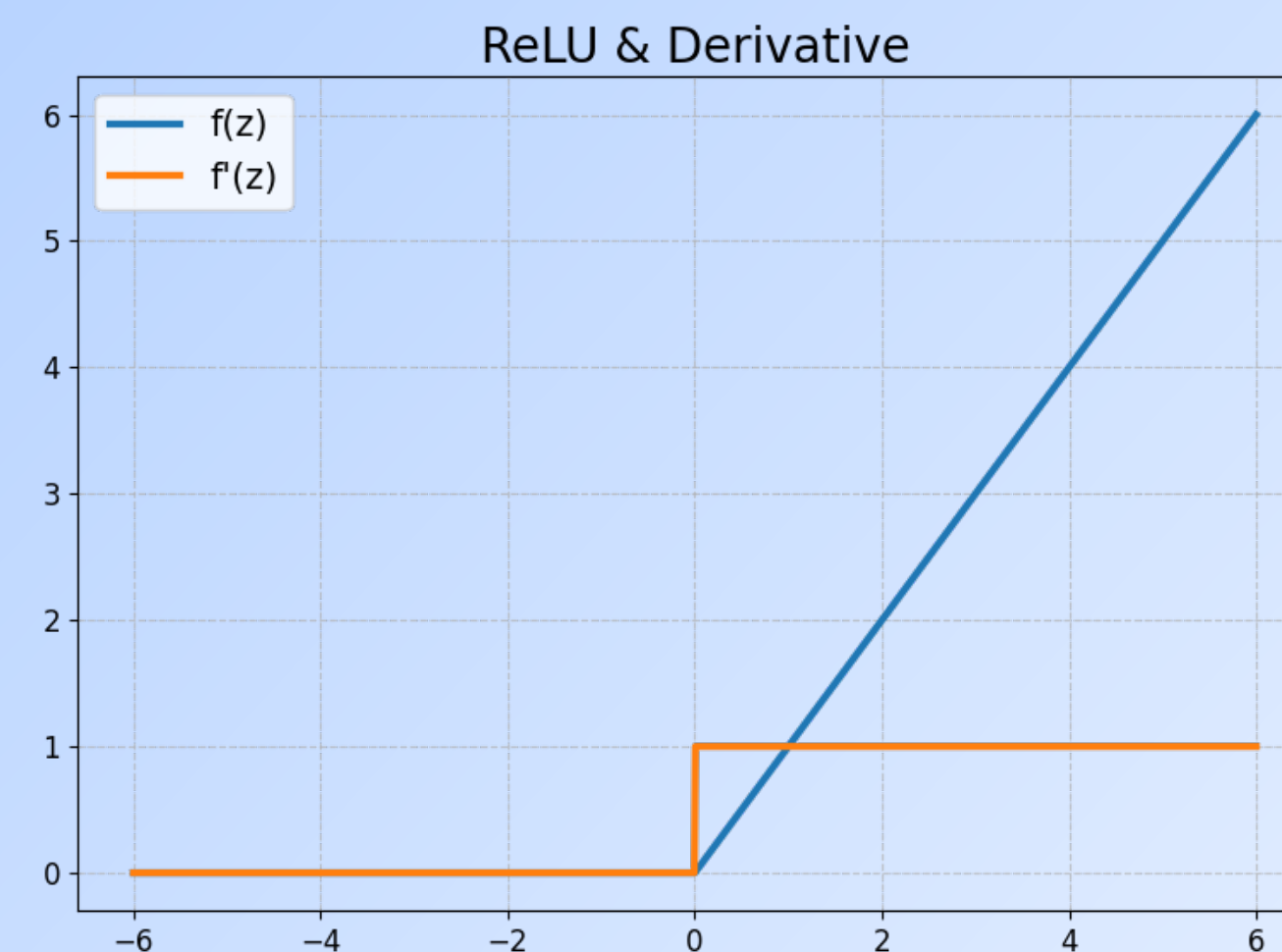
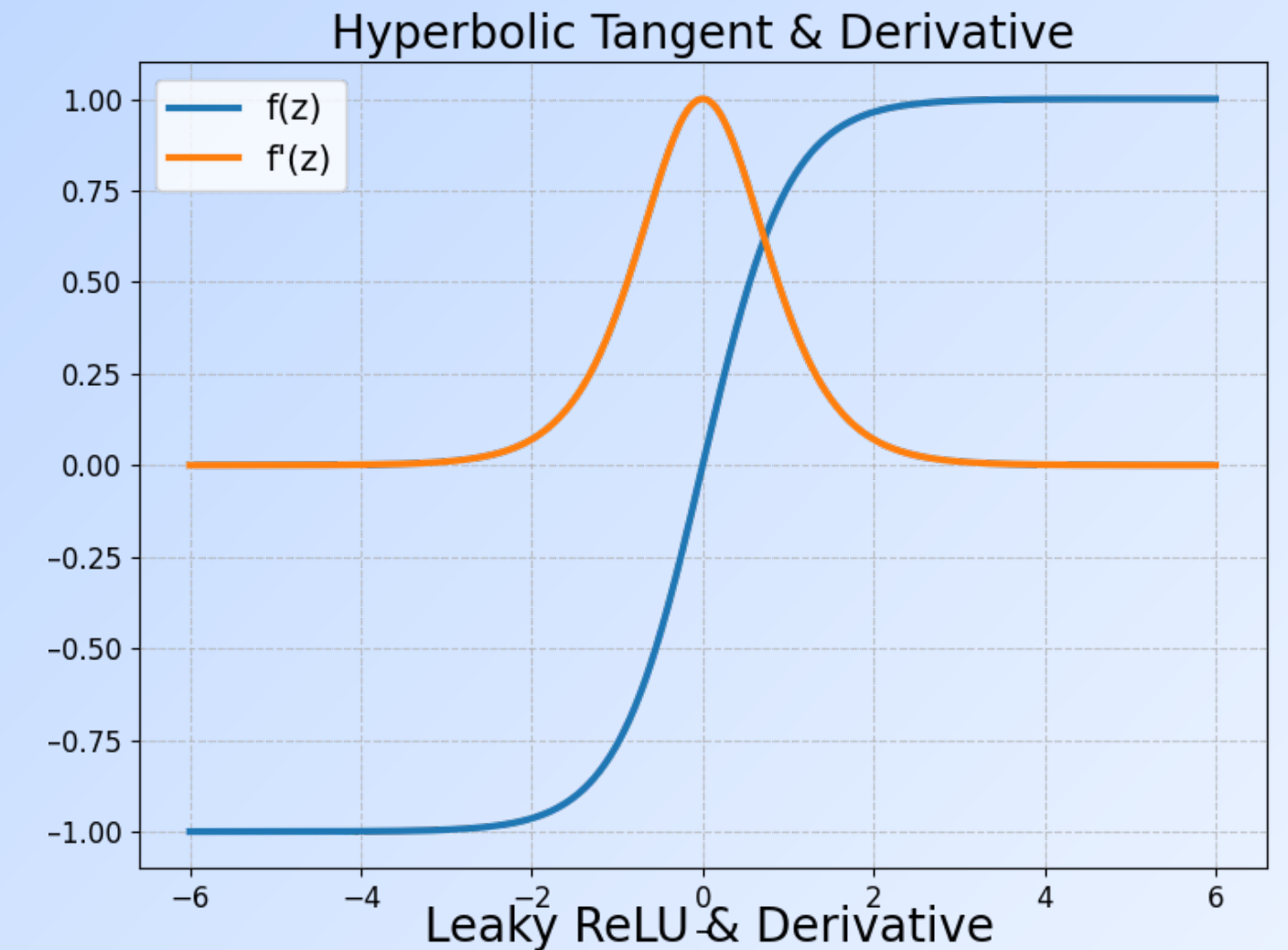
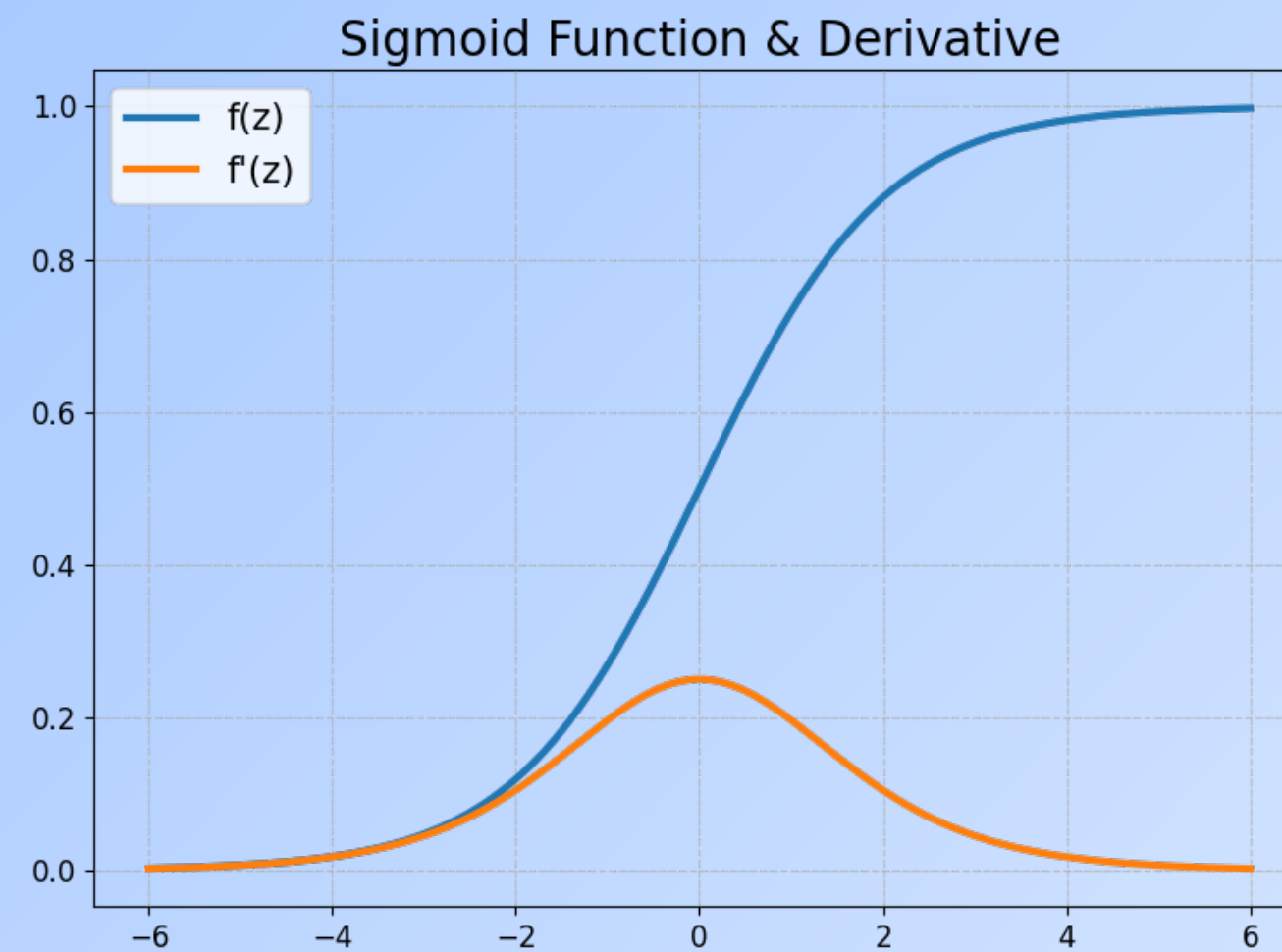
Overcoming Vanishing and Exploding Gradients

- Using non-saturating activation functions
- Weight initialization
- Batch normalization
- Gradient clipping
- Residual connections

Overcoming Vanishing and Exploding Gradients

Solution 0: Choice of Activation Function

- The sigmoid and tanh activation functions cause vanishing gradients due to small magnitudes of their derivatives over most of their active regions.
- ReLU and its variations (such as leaky ReLU, ELU, soft plus etc) mitigate vanishing gradients.
- ReLU can however lead to dead neurons.



Overcoming Vanishing and Exploding Gradients

Solution 1: Appropriate weight initialization

So far, we assumed that the weights of the different layers are randomly initialized.

Let $z = \sum_{i=1}^N w_i x_i$ where the inputs x_i s are iid having 0 mean while the weights are initialized randomly such that $w_i \sim \mathcal{N}(0,1)$ are also iid.

Thus, $Var(z) = \sum_{i=1}^N Var(x_i)Var(x_i) = NVar(x_i)$

Thus, $Var(z) \geq NVar(x_i)$ when $Var(w_i) \geq 1$

To ensure that $Var(z) = Var(x_i)$, we need to initialize the weights such that $Var(w_i) = \frac{1}{N}$

This is the idea behind the Xavier initialization.

Overcoming Vanishing and Exploding Gradients

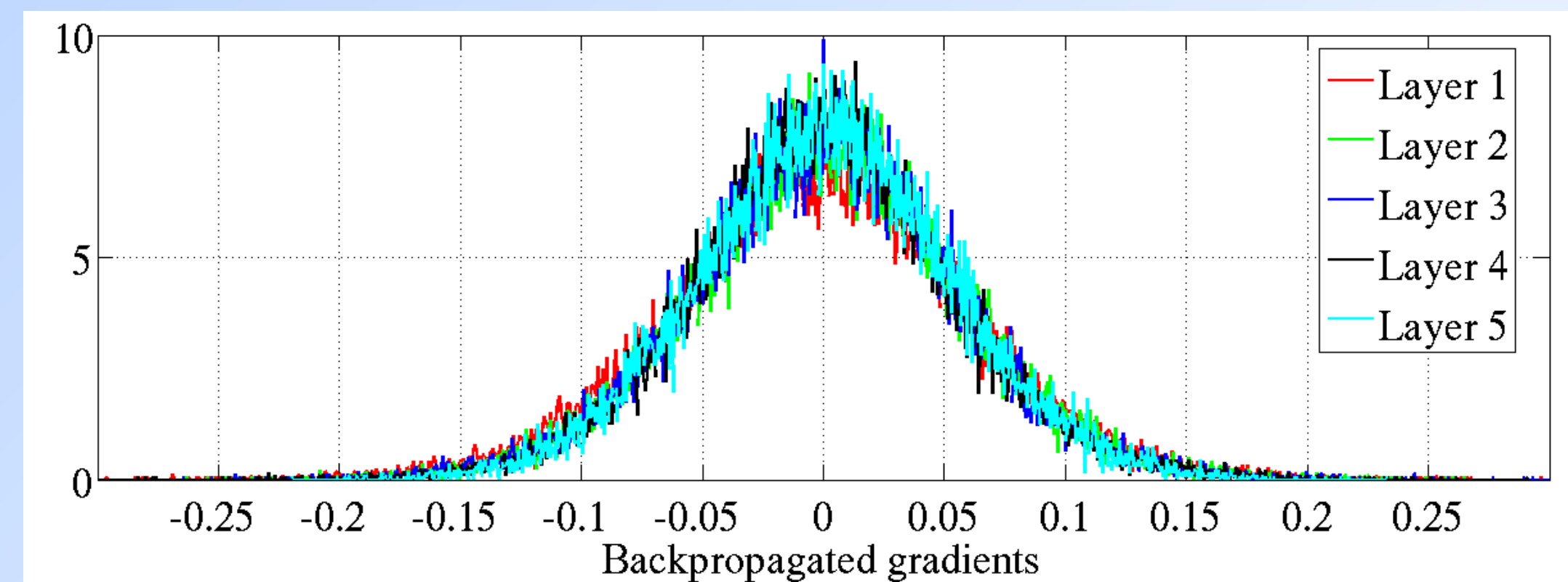
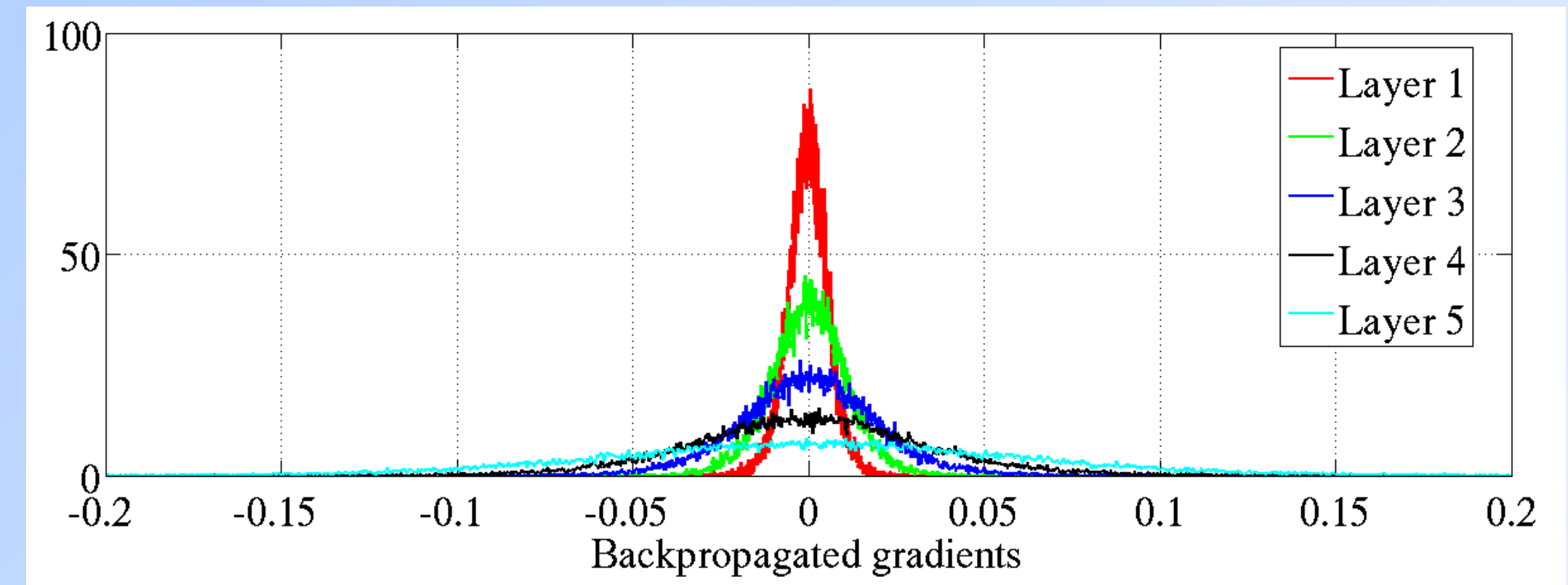
Solution 1: Appropriate weight initialization

Xavier Initialization (Glorot and Bengio, 2010): proposed for tanh activation, primary goal is to maintain variance of the activations (and gradients) across layers.

$$w_i \sim \mathcal{U} \left[-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}} \right]$$

$w_i \sim \mathcal{N} \left(0, \frac{2}{n_{in} + n_{out}} \right)$, where n_{in} and n_{out} are the number of inputs and outputs of the neuron.

He Initialization (He et al., 2015): For ReLU activation, He initialization suggests using $Var(w_i) = \frac{2}{n_{in}}$



Histograms of backpropagated gradients across different layers with random weight initialization (top) and Xavier initialization (bottom)

Overcoming Vanishing and Exploding Gradients

Solution 2: Batch normalization (Ioffe and Szegedy, 2015)

The goal of batch normalization is to scale the intermediate features computed by each hidden unit such that:

- The mean and standard deviation over each mini batch are β_i and γ_i respectively for the i^{th} hidden unit.
- β_i and γ_i are learnable parameters

Consider a minibatch $\mathcal{B} = \{z_i^{(1)}, \dots, z_i^{(m)}\}$ of activations computed as inputs to the i^{th} hidden unit.

$$\mu_i = \frac{1}{m} \sum_{j=1}^m z_i^{(j)} \text{ and } \sigma_i^2 = \frac{1}{m} \sum_{j=1}^m (z_i^{(j)} - \mu_i)^2 \quad (\text{minibatch mean and variance})$$

$$\bar{z}_i^{(k)} = \frac{z_i^{(k)} - \mu_i}{\sigma_i + \epsilon} \text{ for } k = 1, 2, \dots, m \quad (\text{standardization; } \epsilon \text{ is a small constant of the order of } 10^{-6})$$

$$\tilde{z}_i^{(k)} = \gamma_i \bar{z}_i^{(k)} + \beta_i \quad (\text{scaling and shifting for batch normalization})$$

Overcoming Vanishing and Exploding Gradients

- Batch normalization serves more than one objective:
 - Controls internal covariate shift within deep networks.
 - Prevents vanishing and exploding gradients by scaling the features computed at each layer.
 - Acts as a form of regularization.
- Batch normalization could be applied pre-activation or post-activation, though pre-activation batch normalization was originally recommended.
- For performing inference on a single test sample with a model employing batch normalization, the population mean and variance from the entire training set are used to perform normalization.

Overcoming Vanishing and Exploding Gradients

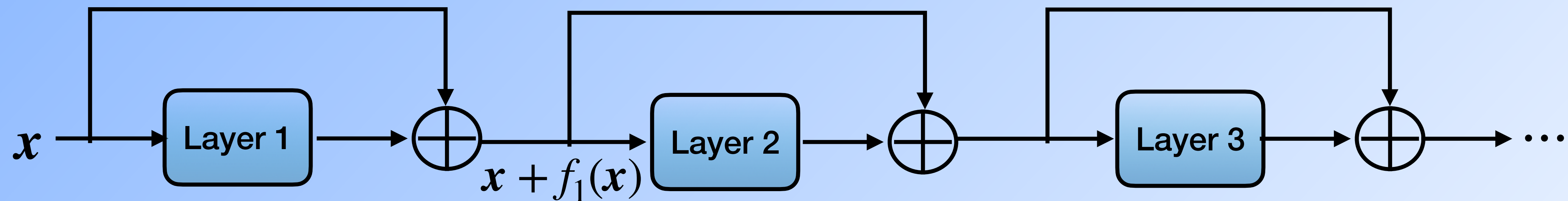
Solution 3: Gradient Clipping

- A straightforward solution to vanishing and exploding gradients involves clipping the magnitudes of the gradients.
- Two commonly used strategies for gradient clipping:
 - Value based clipping: maximum and minimum thresholds are imposed on the values of the partial derivatives that constitute the gradient.
 - Norm based clipping: the gradient vector is normalized using its L_2 norm.
- Gradient clipping is primarily used to avoid exploding gradients in recurrent neural networks.

Overcoming Vanishing and Exploding Gradients

Solution 4: Residual connections

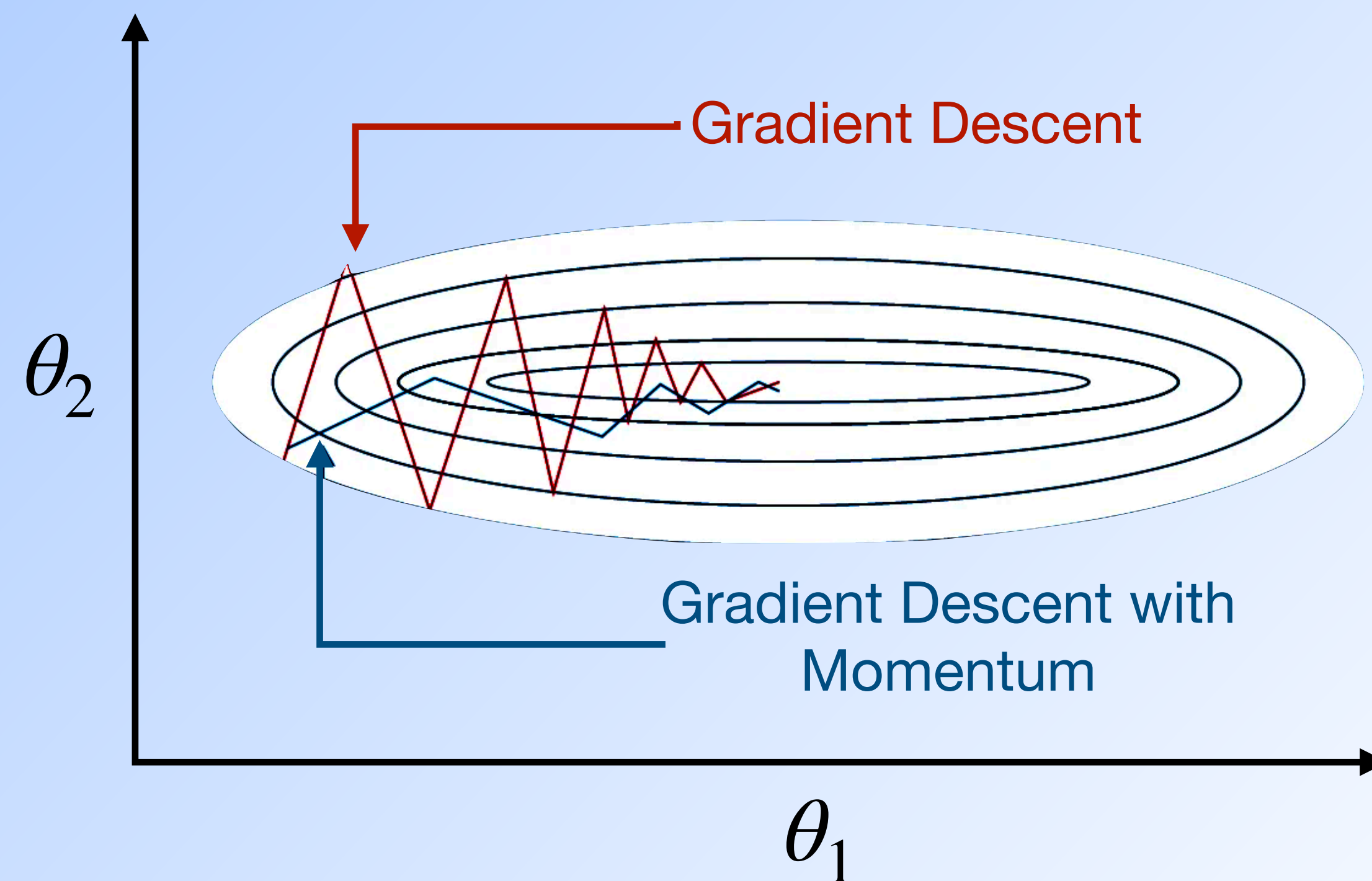
- The idea of residual or skip connections is to provide additional paths of data flow from the earlier to later layers of a deep architecture by skipping intermediate layers.



- Residual connections thus provide many paths of varying lengths for the input signal to propagate forward as well as for the gradient signal to propagate backward.
- Networks having residual connections could be considered as ensemble of shallow and deep networks, where gradients can take shorter paths to flow back to earlier layers, thereby mitigating the vanishing/exploding gradient problems.

Gradient Descent with Momentum

- While navigating an uneven loss surface with local minima, flat regions and saddle points, gradient descent is often prohibitively slow and may even fail to converge to a good solution.
- Momentum based gradient accelerates learning by introducing the notion of velocity, which is set to an exponentially decaying average of the past gradients.
- The key idea behind momentum based methods is to favor the consistent direction of descent over multiple update steps.



Gradient Descent with Momentum

Consider the parameter vector θ needs to be updated using momentum based gradient descent. Then the update rule is given by:

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla_{\theta} J(\theta)$$

$$\theta \leftarrow \theta + \mathbf{v}$$

Here, $\beta \in [0,1]$ is the momentum parameter that determines the contribution of the previous gradients, while α is the learning rate.

Gradient Descent with Momentum

Nesterov Momentum

Nesterov momentum is a modified version of the standard momentum based gradient descent, where a correction is applied as follows:

$$\mathbf{v} \leftarrow \beta \mathbf{v} - \alpha \nabla_{\theta} J(\boldsymbol{\theta} + \beta \mathbf{v})$$

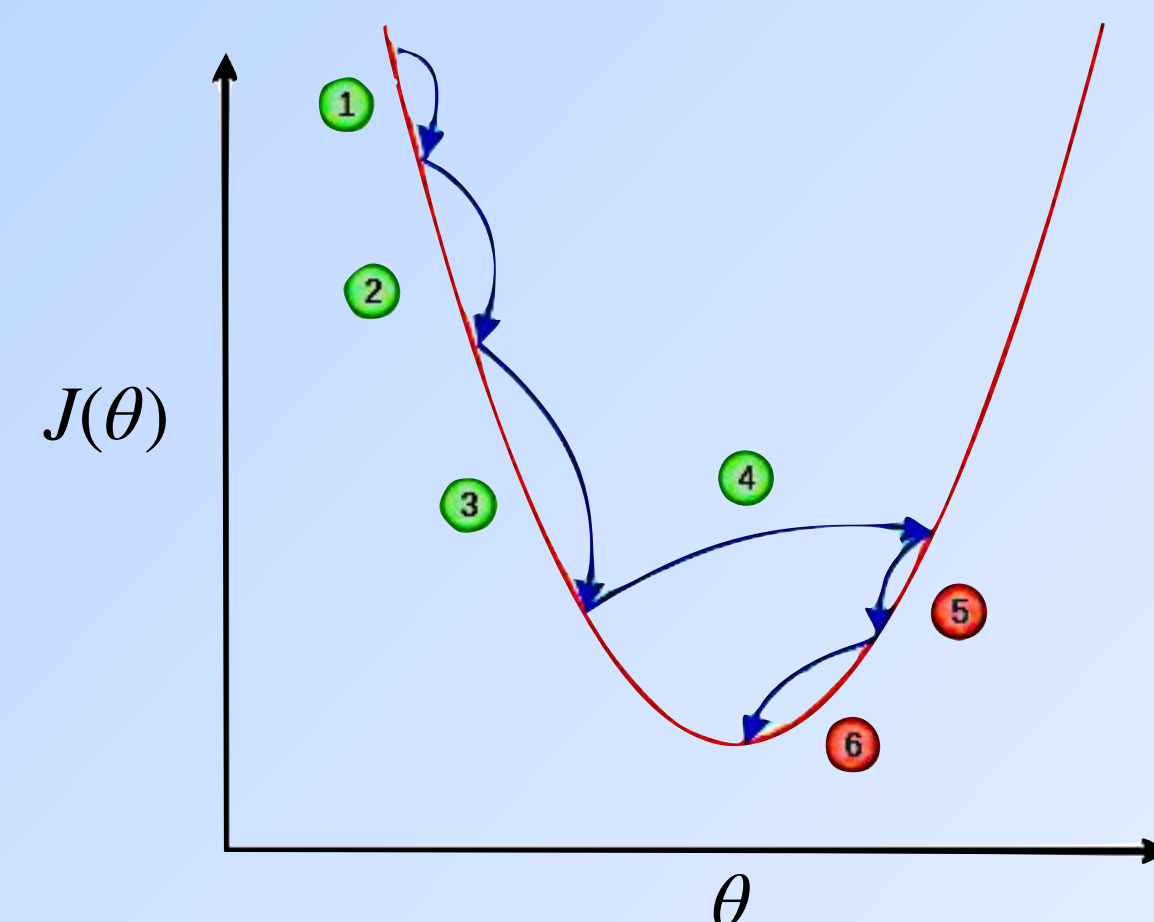
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{v}$$

i.e. the gradient computation takes into account the change due to the momentum portion of the update.

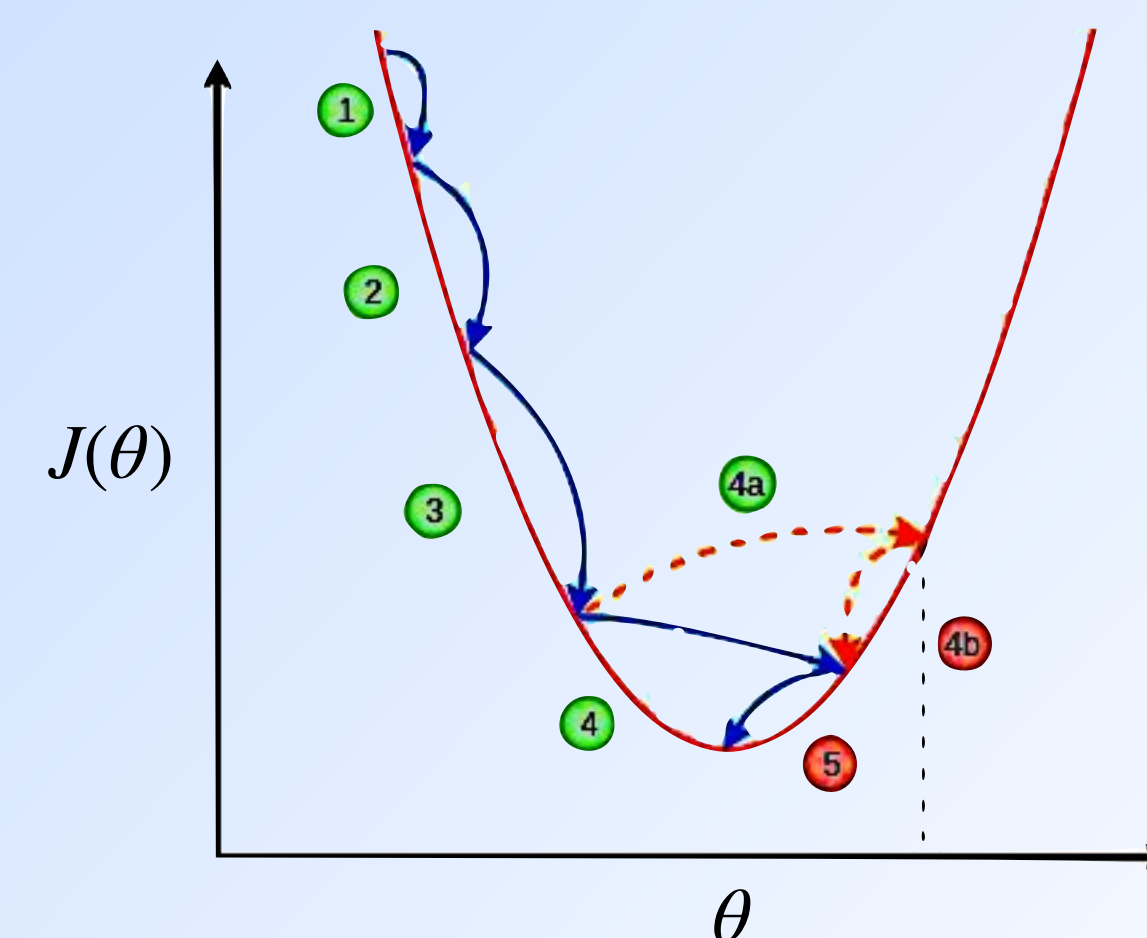
By employing this form of *lookahead*, Nesterov momentum ensures faster convergence as compared to the standard momentum based method.

Nesterov momentum does not work well for stochastic gradient descent or for very small batch sizes.

Gradient Descent with Momentum



Gradient Descent with Nesterov Momentum



Gradient Descent with Adaptive Learning Rates

- The goal of finding a consistent direction of descent over the loss surface can alternatively be accomplished by using different learning rates for different model parameters.
- Large partial derivatives often causes the corresponding parameters to oscillate while those with smaller partial derivatives causes the corresponding parameters to move in a consistent direction.

Gradient Descent with Adaptive Learning Rates

Adagrad (Duchi, 2011)

- The idea behind the Adagrad algorithm is to aggregate the squared magnitudes of the partial derivatives over all the previous training steps, and use this aggregate to scale the learning rate for each parameter at the current step.
- At each step, the square of the gradient magnitude is accumulated and parameter update is performed as follows:

$$A_i \leftarrow A_i + \left(\frac{\partial J}{\partial \theta_i} \right)^2 \quad \forall i \quad (\text{where, } i \text{ is the parameter index})$$

$$\theta_i \leftarrow \theta_i - \frac{\alpha}{\sqrt{A_i + \epsilon}} \left(\frac{\partial J}{\partial \theta_i} \right) \quad \forall i \quad (\text{where } \epsilon \text{ is a small constant of the order of } 10^{-6})$$

Thus, Adagrad encourages faster progress along gentle slopes of the parameter space.

Drawbacks:

- The learning rate becomes progressively smaller leading to very slow progress.
- Makes no distinction between distant past and recent past while scaling the learning rate.

Gradient Descent with Adaptive Learning Rates

RMSProp

RMSProp avoids the premature decay of the learning rate that occurs in Adagrad by introducing an exponentially weighted moving average of the squared gradients.

The parameter update rule given by the RMSProp algorithm is as follows:

$$A_i \leftarrow \rho A_i + (1 - \rho) \left(\frac{\partial J}{\partial \theta_i} \right)^2 \quad \forall i$$

Where $\rho \in [0,1]$ is the decay factor.

$$\theta_i \leftarrow \theta_i - \frac{\alpha}{\sqrt{A_i + \epsilon}} \left(\frac{\partial J}{\partial \theta_i} \right) \quad \forall i$$

The above update rule also suppresses the contribution of squared gradients from the distant past in the scaling of the learning rate.

Gradient Descent with Adaptive Learning Rates

Adadelta (Zeiler, 2011)

Adadelta avoids the need for a global learning parameter by computing it as a function of the incremental updates in the previous steps.

$$\theta_i \leftarrow \theta_i - \sqrt{\frac{\delta_i + \epsilon}{A_i + \epsilon}} \left(\frac{\partial J}{\partial \theta_i} \right) \quad \forall i$$

Let us define $\Delta\theta_i = \sqrt{\frac{\delta_i + \epsilon}{A_i + \epsilon}} \left(\frac{\partial J}{\partial \theta_i} \right)$

Then δ_i accumulates the incremental updates after every step as:

$$\delta_i \leftarrow \rho\delta_i + (1 - \rho)(\Delta\theta_i)^2 \quad \forall i$$

Adadelta removes the necessity of a global learning rate by deriving an approximation of the second derivative of the cost with respect to the parameters.

Gradient Descent with Adaptive Learning Rates

Adam (Kingma and Ba, 2014)

Adam combines the ideas of RMSProp and momentum based updates.

As in the case of RMSProp, it computes the exponentially averaged value of the squared gradients from previous steps:

$$A_i \leftarrow \rho_1 A_i + (1 - \rho_1) \left(\frac{\partial J}{\partial \theta_i} \right)^2 \quad \forall i$$

As in the case of momentum based GD, it computes the exponentially averaged value of the first order gradients from previous steps:

$$F_i \leftarrow \rho_2 F_i + (1 - \rho_2) \left(\frac{\partial J}{\partial \theta_i} \right) \quad \forall i$$

A bias correction is applied to A_i and F_i at each time step t to account for their initialization with 0 values.

$$\hat{A}_i = \frac{A_i}{1 - \rho_1^t}$$

$$\hat{F}_i = \frac{F_i}{1 - \rho_2^t}$$

Gradient Descent with Adaptive Learning Rates

As $t \rightarrow \infty$, $\hat{A}_i \rightarrow A_i$ and $\hat{F}_i \rightarrow F_i$

Then the update rule for parameter θ_i at step t is given by:

$$\theta_i \leftarrow \theta_i - \frac{\alpha}{\sqrt{\hat{A}_i + \epsilon}} \hat{F}_i \quad \forall i$$

Adam incorporates the advantages of several optimization algorithms and offers a robust choice for training deep neural networks.

